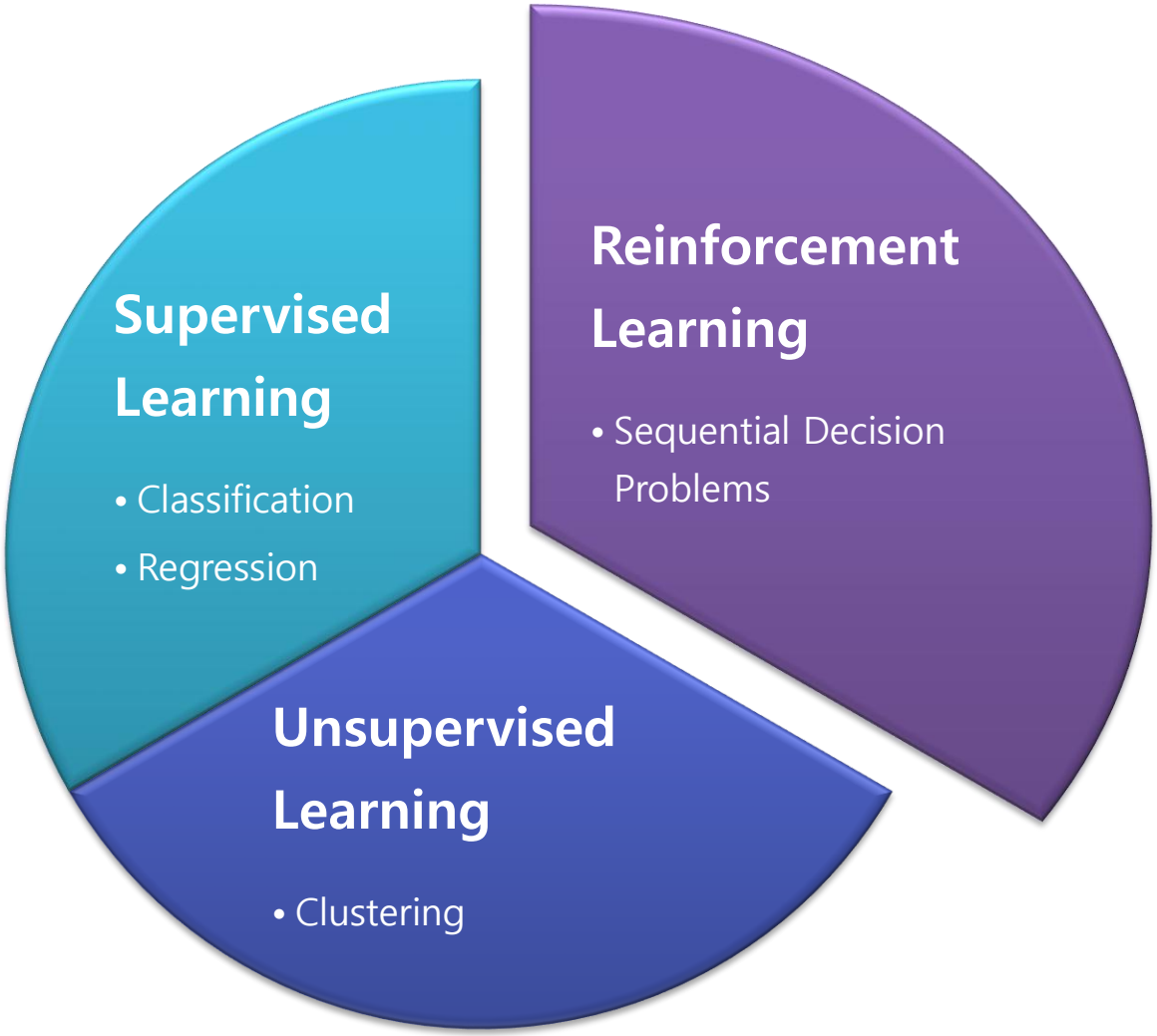
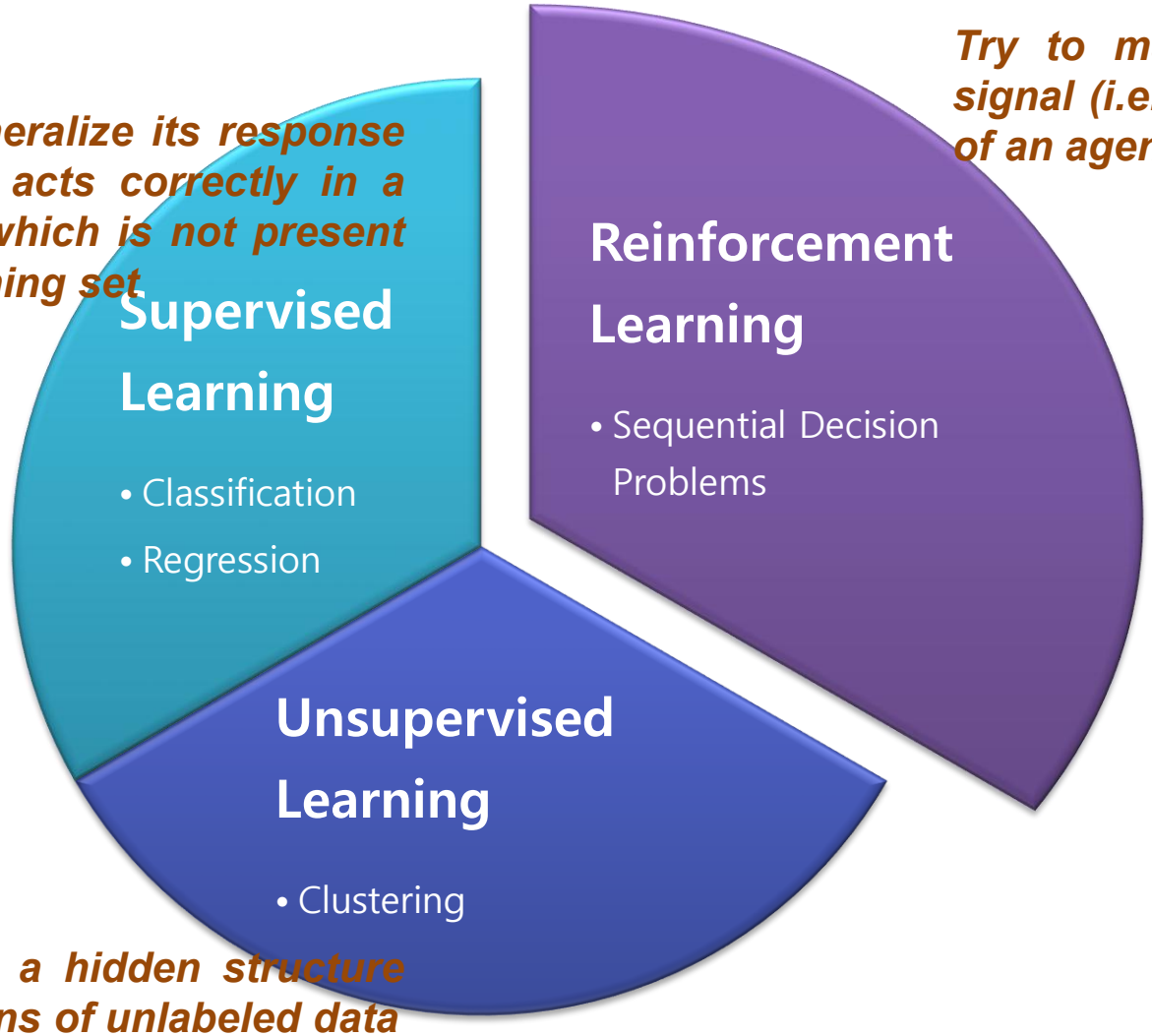
Reinforcement Learning

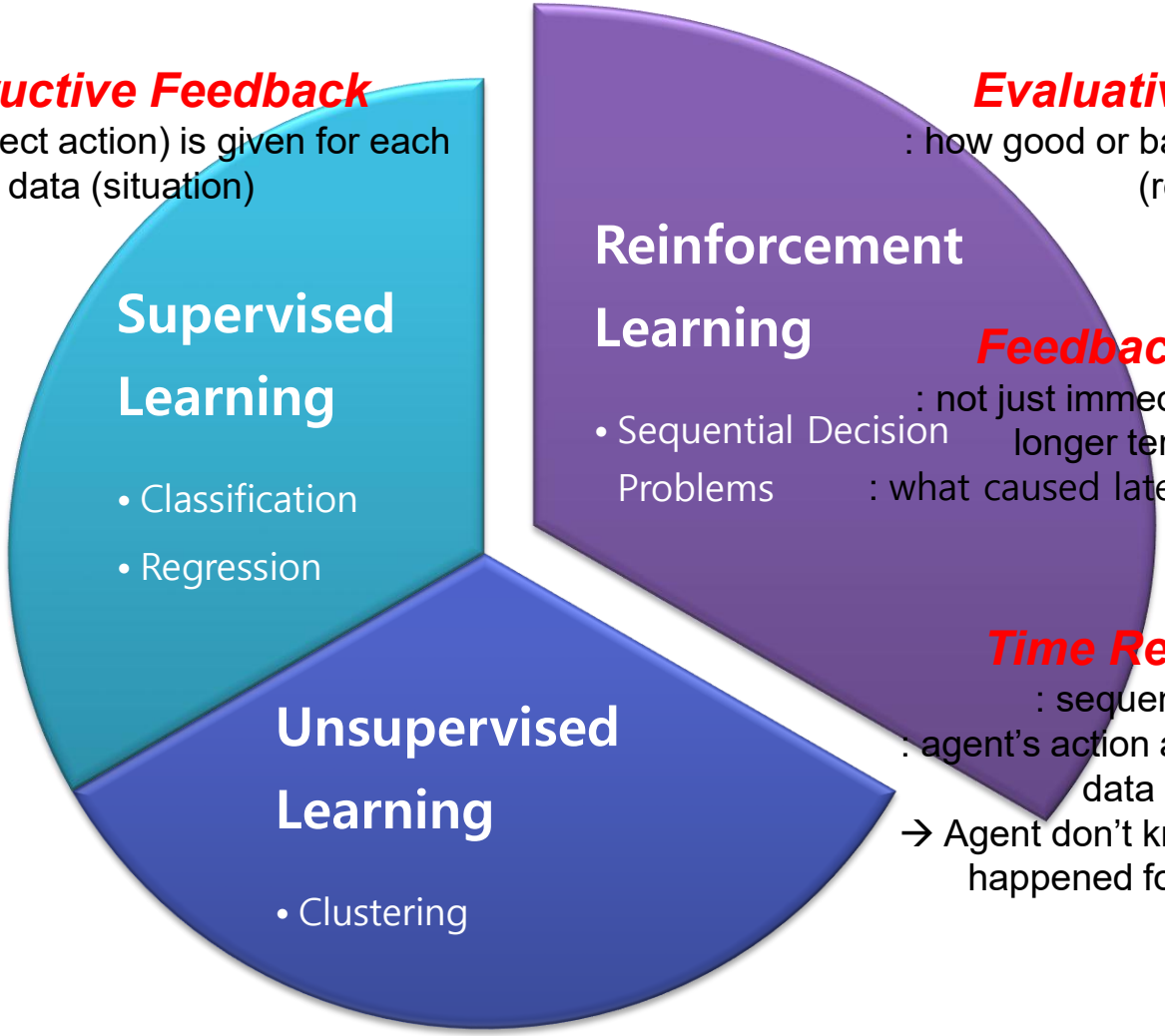
ML Types


Supervised Learning
• Classification
• Regression
Reinforcement Learning
• Sequential Decision Problems
Unsupervised Learning
• Clustering

Goal


Try to generalize its response so that it acts correctly in a situation which is not present in the training set
Supervised Learning
• Classification
• Regression
Try to maximize a reward signal (i.e., achieve the goal of an agent)
Reinforcement Learning
• Sequential Decision Problems
Unsupervised Learning
• Clustering
Try to find a hidden structure in collections of unlabeled data

Types of Feedback



Instructive Feedback

: label (correct action) is given for each data (situation)

Evaluative Feedback

: how good or bad is the action taken (reward)

Feedback is delayed

: not just immediate reward but also longer term ramification

: what caused later high or low rewards?

Time Really Matters

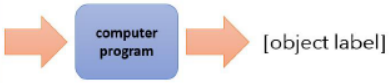
: sequential (not i.i.d)

: agent's action affect the subsequent data it receives

→ Agent don't know what would have happened for other decisions

Supervised Learning





input: $\mathbf{x}$

output: $\mathbf{y}$

data: $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$

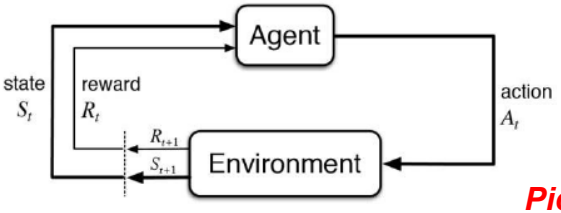
goal: $f_\theta(\mathbf{x}_i) \approx \mathbf{y}_i$

$p_\theta(\mathbf{x})$ $p_\theta(\mathbf{y}|\mathbf{x})$

Someone gives this to you

Output is thinkable (doesn't try to do better than the data)

Reinforcement Learning



input: $\mathbf{s}_t$ *at each time step*

output: $\mathbf{a}_t$ *at each time step*

data: $(\mathbf{s}_1, \mathbf{a}_1, r_1, \ldots, \mathbf{s}_T, \mathbf{a}_T, r_T)$

goal: learn $\pi_\theta : \mathbf{s}_t \rightarrow \mathbf{a}_t$

to maximize $\sum_t r_t$

Pick your own actions

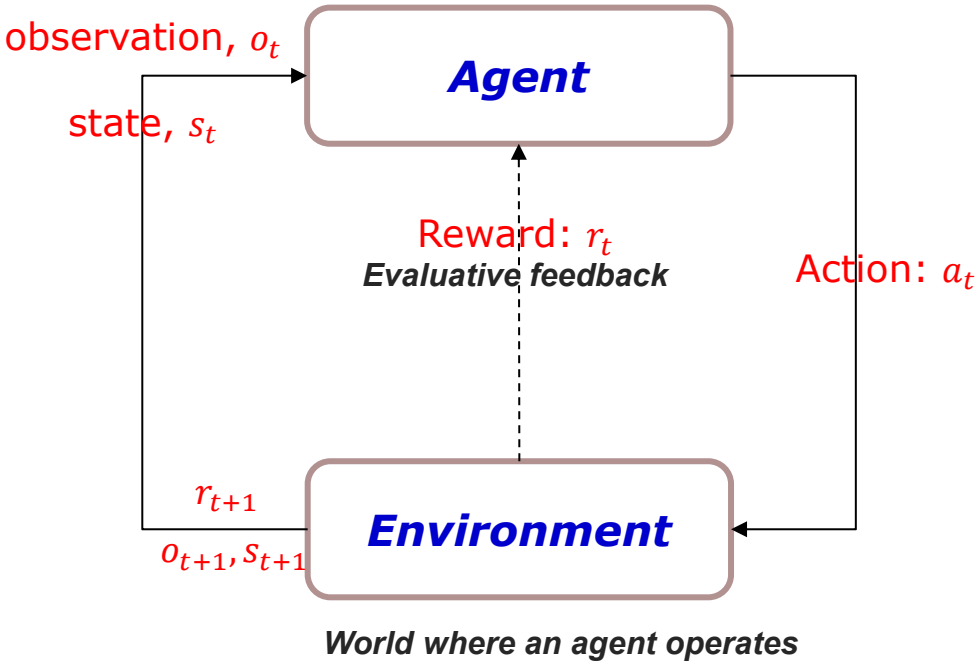
What is a loss function?

Output may be unthinkable (discover new solutions)

RL Components

- At each step t,
- The agent:
  - Executes action a_t
  - Receives observation o_t
  - Receives scalar reward r_t
- The environment:
  - Receives action a_t
  - Emits observation o_{t+1}
  - Emits scalar reward r_{t+1}
- Trajectory (History, Episode)
  - Set of (state, action, reward) tuples

- ***Observation: a situation an agent perceives***
- ***State: information used to determine what happens next***



World where an agent operates

Total Reward (Return) $R_t = \sum_{i=t}^{\infty} r_t = r_t + r_{t+1} + \cdots$

Discounted Total Reward (Return) $R_t = \sum_{i=t}^{\infty} \gamma^i r_t = \gamma^t r_t + \gamma^{t+1} r_{t+1} + \cdots$ **(discounting factor: $0 < \gamma < 1$)**

Examples

Agent State: $S_t = f(H_t)$

(agent can get only the observations and rewards)







Observation O_t

An output of env. Based on its state

Action

Environment

Env. State: $S_t(env)$

(Ex. numbers in the console box)

Examples

- What would be a state (or observation), action, trajectory, and reward?



Robot



Chatbot



state $\mathbf{s}$ - RGB images, joint positions, joint velocities

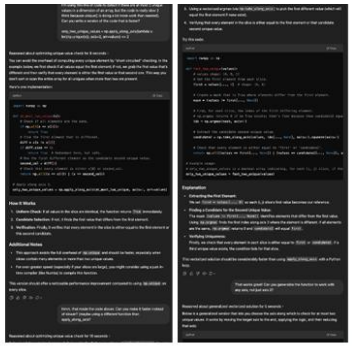
action $\mathbf{a}$ - commanded next joint position

trajectory $\boldsymbol{\tau}$ - 10-sec sequence of camera, joint readings, controls at 20 Hz

$$(\mathbf{s}_1, \mathbf{a}_1, \mathbf{s}_2, \mathbf{a}_2, \ldots, \mathbf{s}_T, \mathbf{a}_T),\ T = 200$$

reward $r(\mathbf{s}, \mathbf{a})$ = 1 if the towel is on the hook in state $\mathbf{s}$

0 otherwise



observation $\mathbf{o}$ - the user's most recent message

action $\mathbf{a}$ - chatbot's next message

trajectory $\boldsymbol{\tau}$ - variable length conversation trace

$$(\mathbf{o}_1, \mathbf{a}_1, \mathbf{o}_2, \mathbf{a}_2, \ldots, \mathbf{o}_T, \mathbf{a}_T)$$

reward $r(\mathbf{s}, \mathbf{a})$ = 1 if the user gives upvote

-10 if the user downvotes

0 if no user feedback

How to drive an agent to make an optimal decision?

Value Learning

- Rank the value of a state or an action taken at a state $(V(s), Q(s,a))$

  Find $\boldsymbol{Q(s,a)}$

- Then, in each state, take the action which has the highest value $Q(s,a)$

  $\boldsymbol{a = \underset{a}{\operatorname{argmax}}\, Q(s,a)}$ for all s

Policy Learning

- Find the optimal policy directly

  Find $\boldsymbol{\pi(s) = \{\pi(a|s)\}}$

- Then, sample an action from the policy (note: the policy is a probability of taking an action a at state s)

  Sample $\boldsymbol{a \sim \pi(s)}$

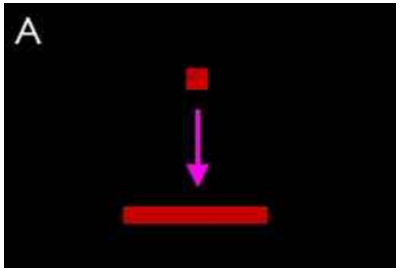
Q-function

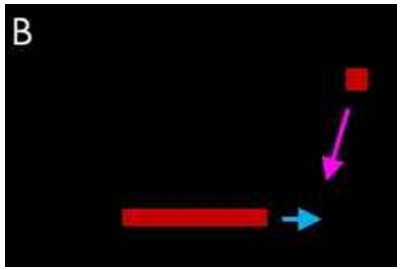
- Recall
  - $R_t = \sum_{i=t}^{\infty} \gamma^i r_t = \gamma^t r_t + \gamma^{t+1} r_{t+1} + \cdots$
- Q-function
  - $Q(s_t, a_t) = E[R_t | s_t, a_t]$
  - Expected total *future* reward an agent in state s can receive by taking an action a
- Then, an agent take an action in each state by selecting an action
  - $a^* = \pi^*(s) = \underset{a}{\operatorname{argmax}}\, Q(s, a)$

How to Estimate Q-function

- It is a difficult task !!
- Example)
  - Which action has a higher Q-value?
    - depends on the state
  - https://github.com/GiannisMitr/DQN-Atari-Breakout

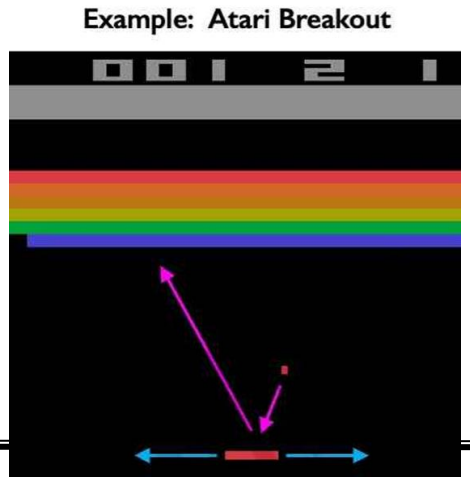
Action: a_1





Action: a_2

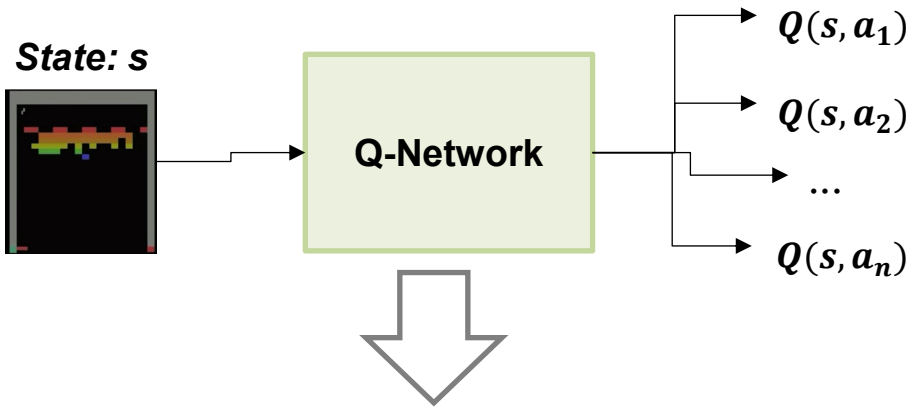
State: s_1





State: s_2

Learn Q-function via Neural Net (DQN)



How can we train the Q-Network?

- Note that our goal is to maximize the total reward (i.e., return)
- The expected ***maximum reward*** we can expect by taking an action a_t in a state s_t at time t is

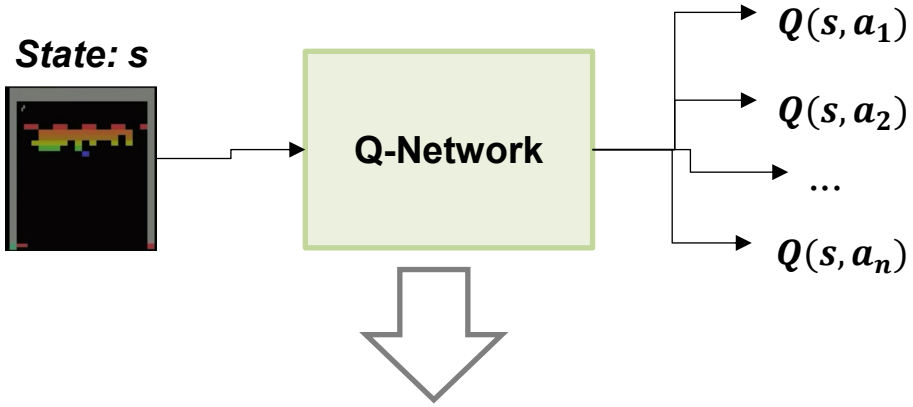
$$r + \gamma \max_{a'} Q(s_{t+1}, a')$$ ← Target value

Reward obtained when taking a_t in a state s_t

Maximum Q-value at the next state s_{t+1}
(i.e., maximum return obtainable at the next state)

$$Q(s_t, a_t)$$ ← Maximum reward predicted by Q-Network

Loss Function



$$Loss = E\left[\left\|\left(r + \gamma \max_{a'} Q(s_{t+1}, a')\right) - Q(s_t, a_t)\right\|^2\right]$$

DQN Drawbacks

- Difficult to handle continuous action spaces

- Deterministic
  - Action is deterministically decided from Q-function
    - It gives the same action all the time
  - But environments are not always deterministic
    - Exploration ?
      - Ex) ε –greedy action selection

DQN

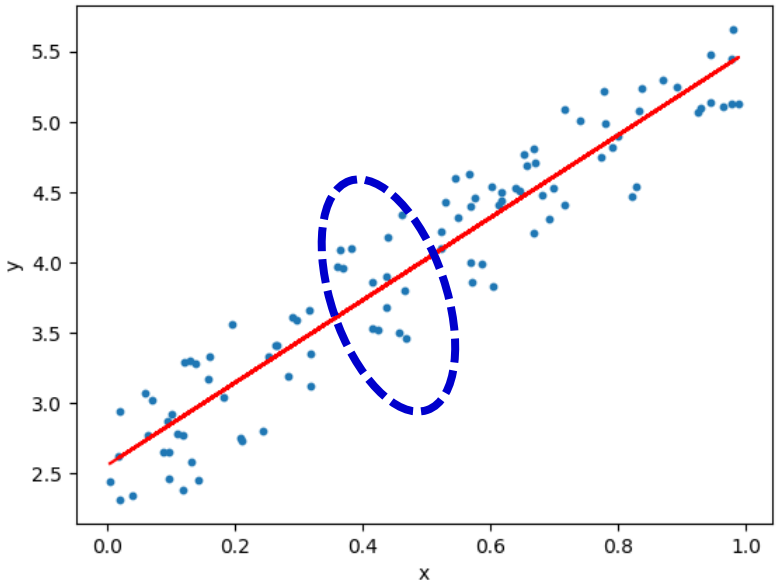
❑ Problem

- Estimate Q-value ($\hat{Q}$) *does not converge* to the optimal Q-value (Q)
- Because
  - Correlations between samples
  - Non-stationary targets
- Solution : ***DQN***
  - ***Replay Memory***
  - ***Network Separation (Train Net, Target Net)***

Problem 1. Correlation between Samples

- Similar Data
  - Unable to infer the characteristics of *whole* dataset





Problem 2. Non-Stationary Target

- Recall the loss function of the Q-Network

We choose (i.e., train) θ so that

$$min_{\{\theta\}} \sum_{t=0}^{T} \left[\hat{Q}(s_t, a_t|\theta) - \left(r_t + \gamma max_{\{a'\}} \hat{Q}(s_{t+1}, a'|\theta) \right) \right]^2$$

신경망 출력 (예측값)

Target

Optimal Q-value (i.e., $Q^*(s,a)$)로 간주 (즉, 현재 action의 reward (r)과 천이한 다음 상태 s'에서 얻을 수 있는 최대 이득의 합)

$$\hat{Y} = \hat{Q}(s_t, a_t|\theta)$$

학습 (i.e., θ 조절)을 통해 $\hat{Y}$ 를 Y와 같게 만드는 것이 Q-Network의 목적

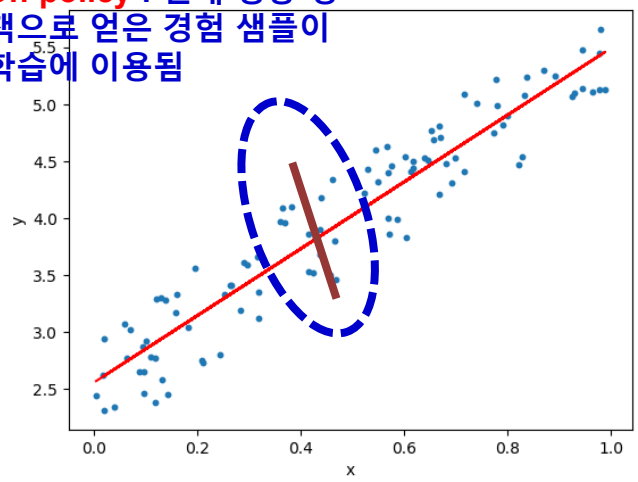
$$Y = r_t + \gamma max_{\{a'\}} \hat{Q}(s_{t+1}, a'|\theta)$$

- Q-Network은 하나의 신경망만 사용
  - 신경망의 출력인 예측값 $\hat{Y}$ 이 Target과 같아지도록 신경망의 Weight θ를 변경하면 차 후 loss 평가에서 Target인 Y값도 변경됨
  - 즉, 이전 target을 맞추기 위해 θ 를 변경했지만 이로 인해 target 값이 바뀌게 되어 (i.e., *non-stationary target*) loss가 줄어들지 않을 수 있음

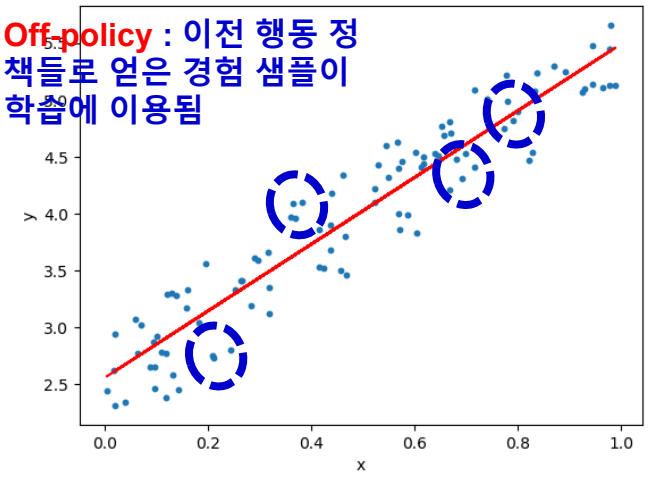
Experience Replay (경험 재현)

- 학습 샘플의 시간 상관관계 (Correlation between samples)로 인한 문제
  - 안 좋은 상황이 지속될 경우 인공신경망이 그 방향으로 계속 update 됨
    - Cf) deep learning은 샘플들이 서로 독립적 (i.i.d)임을 가정
- Replay Memory
  - 매 time step 마다 환경에서 얻은 샘플 (S, A, R, S')을 replay memory에 저장
    - 즉, 많은 에피소드에서 얻은 경험을 저장
  - 학습이 필요할 때 (즉, 신경망을 update할 때) Reply memory에서 무작위로 여러 개의 샘플을 추출하여 인공신경망 update
    - 샘플들의 시간 상관관계 무력화
    - 단일 샘플이 아닌 여러 샘플로 인공신경망 update하여 학습이 안정적 (gradient variance 감소)
    - 즉, 현재 정책에 의한 샘플이 아닌 과거 정책들에 의한 샘플로 학습 → Off-Policy (Q-learning)

On-policy : 현재 행동 정책으로 얻은 경험 샘플이 학습에 이용됨



Off-policy : 이전 행동 정책들로 얻은 경험 샘플이 학습에 이용됨



DQN Algorithm (2013)

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$
  end for
end for

전처리 ϕ(s)가 없는 경우 $\phi_1 = s$

epsilon-greedy exploration

step in the environment

replay buffer and sampling

regression target

gradient step (학습)

Target Q value (target == label)

신경망이 근사한 Q value

Target Network

- DQN 문제점 : Moving Target
  - target function $Q^{\pi}_{tar:DQN}(s,a) = R_{t+1} + \gamma \max_{a' \in A} Q(S_{t+1}, A_{t+1}|\theta)$이 신경망의 추정값 $(Q(S_{t+1}, A_{t+1}|\theta))$에 의해 결정됨
  - $Q(s,a|\theta)$를 학습하는 순간 (i.e., θ 변경) target 값도 변하여 학습 성능 저하됨
- Target Network
  - DQN과 동일한 신경망이지만 신경망의 parameters 값 ($\boldsymbol{\theta^-}$)은 가끔씩만 update
    - ($\boldsymbol{\theta^-}$ 는$\boldsymbol{\theta}$ 의 현재값으로 주기적으로 갱신됨 (즉, 주기 T_{update}마다 $\boldsymbol{\theta^- = \theta}$)
    - 즉, 일정시간 동안 Target (정답)을 만들어내는 인공신경망을 유지
    - $Q^{\pi}_{tar:DQN}(s,a)$ 을 계산할때 사용
- Q-value update rule
  - DQN

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R_{t+1} + \gamma \max_{a' \in A(s')} Q(s', a'|\theta) - Q(s,a|\theta)\right]$$

  - With Target Network

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R_{t+1} + \gamma \max_{a' \in A(s')} \boldsymbol{Q(s', a'|\theta^-)} - \boldsymbol{Q(s,a|\theta)}\right]$$

DQN with Target Network

Initialize network Q
Initialize target network $\hat{Q}$
Initialize experience replay memory D
Initialize the *Agent* to interact with the Environment
while *not converged* **do**

  `/* Sample phase`
  $\epsilon \leftarrow$ setting new epsilon with ϵ-decay
  Choose an action a from state s using policy ϵ-greedy(Q)
  Agent takes action a, observe reward r, and next state s'
  Store transition $(s, a, r, s', done)$ in the experience replay memory D

  if *enough experiences in* D **then**
    `/* Learn phase`
    Sample a random *minibatch* of N transitions from D
    for *every transition* $(s_i, a_i, r_i, s'_i, done_i)$ *in minibatch* **do**
      if $done_i$ **then**
        $y_i = r_i$
      else
        $y_i = r_i + \gamma \max_{a' \in \mathcal{A}} \hat{Q}(s'_i, a')$
      end
    end
    Calculate the loss $\mathcal{L}$ =1/N $\sum_{i=0}^{N-1}(Q(s_i, a_i) - y_i)^2$
    Update Q using the SGD algorithm by minimizing the loss $\mathcal{L}$
    Every C steps, copy weights from Q to $\hat{Q}$
  end
end

Target function

Target network update

Value Learning

- Rank the value of a state or an action taken at a state $(V(s), Q(s,a))$

  Find $\boldsymbol{Q(s,a)}$

- Then, in each state, take the action which has the highest value $Q(s,a)$

  $\boldsymbol{a = \underset{a}{\operatorname{argmax}}\, Q(s,a)}$ for all s

Policy Learning

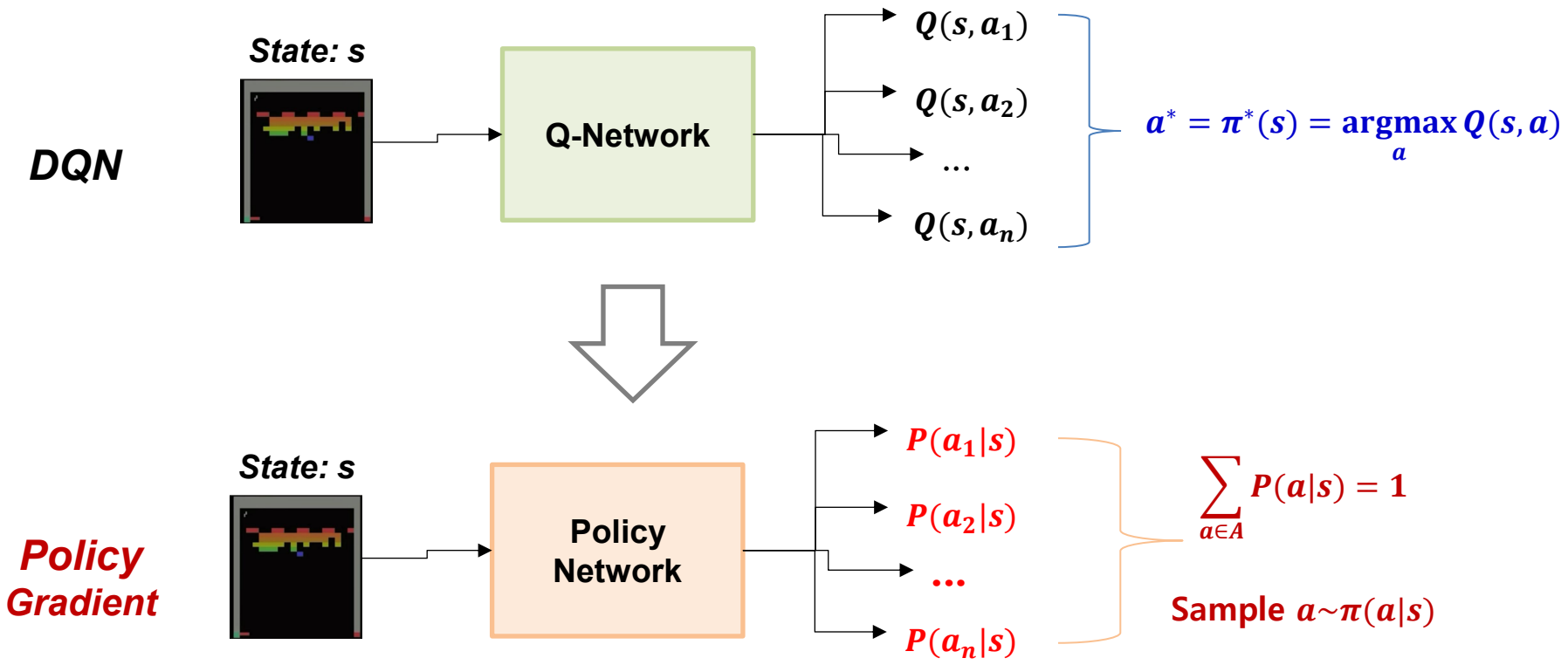
- Find the optimal policy directly

  Find $\boldsymbol{\pi(a|s)}$

- Then, sample an action from the policy (note: the policy is a probability of taking an action s at state s)

  Sample $\boldsymbol{a \sim \pi(a|s)}$

From Q-function to Policy Function



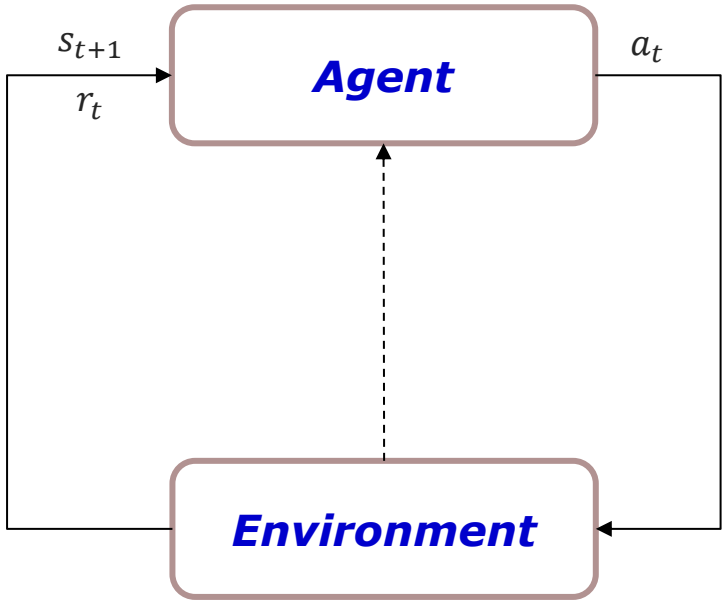
How to Train Policy Gradient? Idea

Training Algorithm

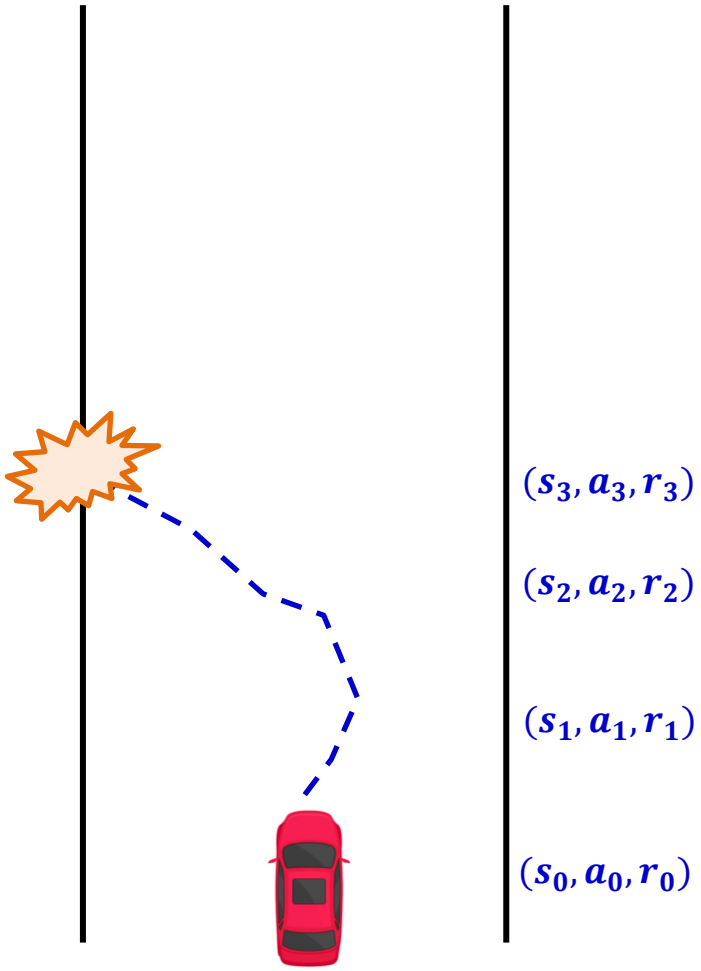
- Initialize an agent
- Repeat:
  - ***Run*** a policy until termination
  - ***Record*** all states, actions, and rewards
  - ***Decrease*** probability of actions that resulted in ***low*** reward
  - ***Increase*** probability of actions that resulted in ***high*** reward

How to Train Policy Gradient? Idea

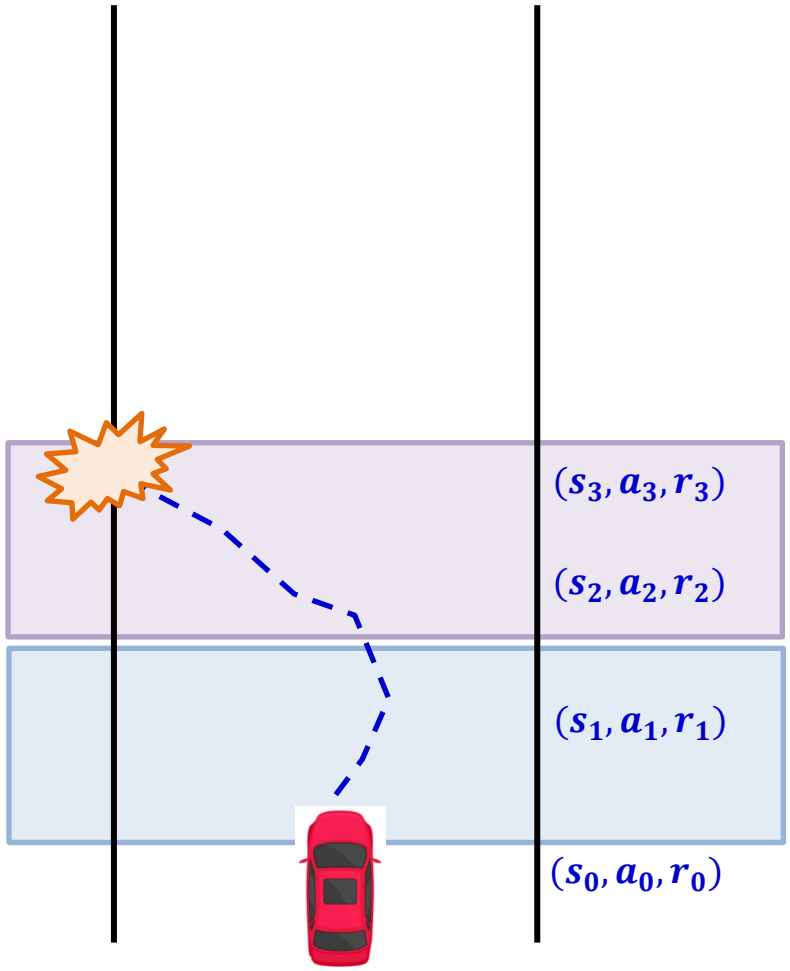
- Example)
  - Self-Driving Car
  - State:
    - Camera, lidar, etc.
  - Action:
    - Steering wheel angle
  - Reward:
    - Distance traveled



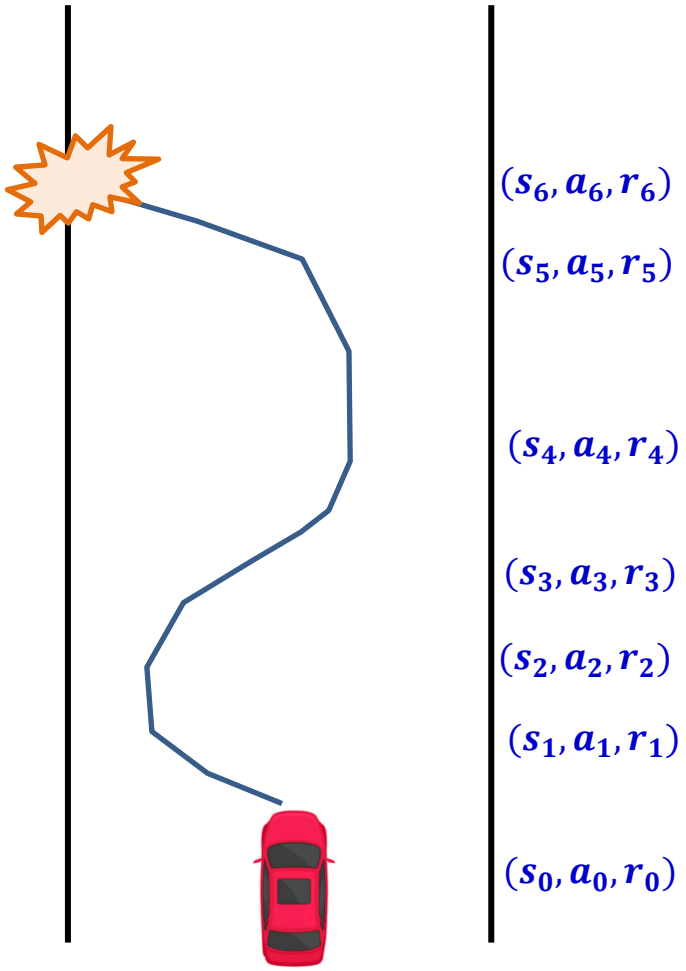
- Initialize agent
- Repeat:
  - ***Run*** a policy until termination
  - ***Record*** all states, actions, and rewards
  - ***Decrease*** probability of actions that resulted in ***low*** reward
  - ***Increase*** probability of actions that resulted in ***high*** reward



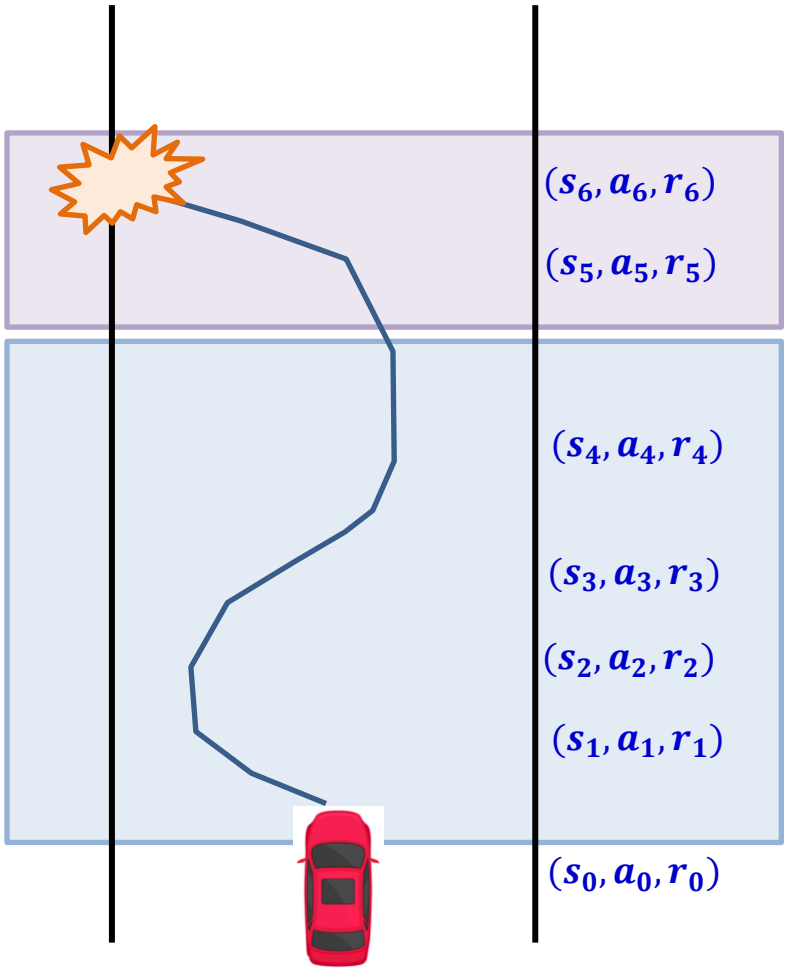
- Initialize agent
- Repeat:
  - ***Run*** a policy until termination
  - ***Record*** all states, actions, and rewards
  - ***Decrease*** probability of actions that resulted in ***low*** reward
  - ***Increase*** probability of actions that resulted in ***high*** reward



- Initialize agent
- Repeat:
  - ***Run*** a policy until termination
  - ***Record*** all states, actions, and rewards
  - ***Decrease*** probability of actions that resulted in ***low*** reward
  - ***Increase*** probability of actions that resulted in ***high*** reward



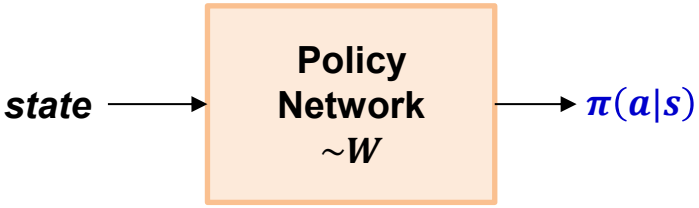
- Initialize agent
- Repeat:
  - ***Run*** a policy until termination
  - ***Record*** all states, actions, and rewards
  - ***Decrease*** probability of actions that resulted in ***low*** reward
  - ***Increase*** probability of actions that resulted in ***high*** reward



Policy Gradient

Training Algorithm

- ❑ Initialize an agent
- ❑ Repeat:
  - ***Run*** a policy until termination
  - ***Record*** all states, actions, and rewards
  - ***Decrease*** probability of actions that resulted in ***low*** reward
  - ***Increase*** probability of actions that resulted in ***high*** reward



Idea: maximize log-likelihood of an action multiplied by reward

$$Loss = -\log P(a_t|s_t)\, R_t = -\log \pi(a_t|s_t)\, R_t$$

Gradient Descent Update

- $W' = W - \nabla loss$
- $W' = W - \nabla \log P(a_t|s_t)\, R_t$

  $= W - \nabla \log \pi(a_t|s_t)\, R_t$

- ***Policy Gradient (REINFORCE)***
- ***Different Policy Gradient RL uses other forms of loss functions***

In case of REINFORCE

$$\nabla \log P(a_t|s_t)\, R_t \approx \nabla_W log\, \pi_W(a_t|s_t)\, Q_\pi(s_t, a_t) \approx \nabla_W log\, \pi_W(a_t|s_t)\, G_t$$

참고) Monte-Carlo Policy-Gradient Method

- $Q_\pi(S_t, A_t)$을 반환값 (i.e., 누적 보상) G_t로 대체
  - 이유: $Q_\pi(S_t, A_t) = E_\pi[G_t|S_t, A_t]$
- Algorithm
  - Episode 동안 지나온 모든 상태들의 반환값 계산
  - Episode 종료 후 model parameter W를 다음과 같이 update

$$W_{t+1} \approx W_t + \alpha\left[\nabla_W log\left(\pi_W(A_t|S_t)\right)G_t\right]$$

- 활성 정책 (on-policy) Algorithm
  - 현재 정책에 의해 생성된 Episode를 이용하여 parameter update

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

- After Training, an agent can drive
- Example
  - https://github.com/idreesshaikh/Autonomous-Driving-in-Carla-using-Deep-Reinforcement-Learning

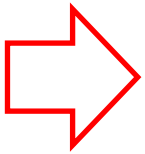




Gathering Data is Difficult

Training Algorithm

- Initialize an agent
- Repeat:
  - ***Run*** a policy until termination
  - ***Record*** all states, actions, and rewards
  - ***Decrease*** probability of actions that resulted in ***low*** reward
  - ***Increase*** probability of actions that resulted in ***high*** reward



Data Gathering Problem

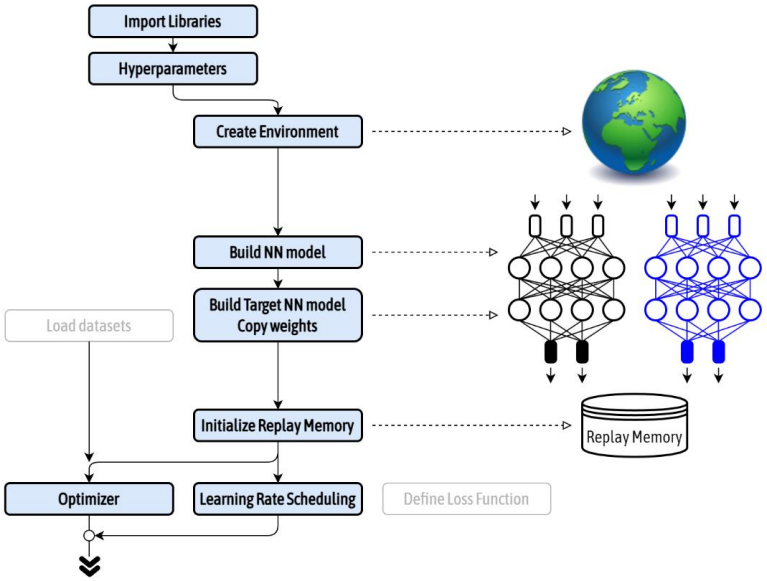
- ***Offline Reinforcement Learning***
- ***Physics Simulation (Sim-to-Real)***
- ***Etc.***

Simulation Environments

- Gymnasium (formerly OpenAI Gym)
  - The industry-standard interface that provides a unified API to connect various simulators to RL algorithms.
  - https://github.com/Farama-Foundation/Gymnasium
- MuJoCo/MuJoCo Playground
  - A physics engine for multi-joint dynamics, frequently used in robotics and manipulation tasks.
  - https://mujoco.org/
- NVIDIA Isaac Lab/Isaac Sim
  - A high-fidelity, GPU-accelerated simulation platform designed for robot learning, featuring realistic physics and sensor simulation for sim-to-real transfer.
- CARLA
  - A widely used open-source simulator for autonomous driving research.
- EnvPool
  - A high-performance, parallel RL environment execution engine that can achieve millions of frames per second.
- PyFlyt
  - Specialized UAV (Unmanned Aerial Vehicle) simulation platform based on the Bullet physics engine.

Exercise: DQN

Algorithm




```python
import gymnasium as gym # Changed from gym
import numpy as np
import torch
import random
from collections import deque
from tqdm import tqdm
import torch.nn as nn
import torch.optim as optim

# PyTorch 기반의 Network 및 DQNAgent 클래스 정의는 여기에 추가

# 환경 설정
env_name = "CartPole-v0"
env = gym.make(env_name)
env.reset(seed=1)  # 재현 가능성을 위해
```

01 : **Input** : Initial Action-value Q-function parameters $Q(s, a, \theta)$
02 : **Target** : Initial Action-value Q-function parameters $Q(s, a, \theta_{targ})$
: Set target parameters equal to main parameters $\theta_{targ} \leftarrow \theta$
03 : **Parameters** : learning rate $\alpha^{\theta} > 0$
04 : Initialize replay memory $\mathcal{D}$ to capacity N
05 : total_step $= 0$, policy_update_freq $= C$
06 : trial_step $= T$
07 : Set max ε and ε decay parameters
08 : **For** episode $= 1, M$ **do**
09 : done $= False$
10 : Reset environment state.
11 : **For** $t = 1, T$ **do** (or **While not** $done$)
12 : With probability ε select a random action a_t
otherwise select $a_t = \underset{a}{\text{argmax}}\, Q(s_t, a; \theta)$
13 : Execute a_t in the environment
14 : Observe next state s_{t+1}, reward r_t, and done signal d_t to indicate whether s_{t+1} is terminal
15 : Store $(s_t, a_t, r_t, s_{t+1}, d_t)$ in replay buffer $\mathcal{D}$
16 : $s_t \leftarrow s_{t+1}$
17 : **If** it's time to update **then**
18 : **For** j in range(however many updates) **do** # mini-batch process
19 : Randomly sample a batch of transitions , $B = (s, a, r, s', d)$ from $\mathcal{D}$
21 : Compute targets (= expected Q)

$$y_j = Q_{\text{expected}} = \begin{cases} r_j, & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max\limits_{a} Q(s'_j, a_j; \theta_{targ}), & \text{otherwise} \end{cases}$$

22 : Reset gradient $d\theta$ to 0
23 : Perform a gradient descent step on $(y_j - Q(s_j, a_j; \theta))^2$

$$\theta \leftarrow \theta + \alpha^{\theta} d\theta$$

24 : **If** total_step % policy_update_freq $= 0$ **then**
25 : Update the target networks with : # hard target update

$$\theta_{targ} \leftarrow \theta$$

26 : Update ε
27 : **End For**
28 : **End If**
29 : **End For**
30 : **End For**
31 :

Algorithm

08 : **For** episode $= 1, M$ **do**
09 : done $= False$
10 : Reset environment state.
11 : **For** $t = 1, T$ **do** $(or$ **While not** $done)$
12 : With probability ε select a random action a_t
otherwise select $a_t = \arg\max_a Q(s_t, a; \theta)$
13 : Execute a_t in the environment
14 : Observe next state s_{t+1}, reward r_t, and done signal d_t
15 :
16 :

```python
# 에이전트 훈련
agent = DQNAgent(env, batch_size)

if __name__ == "__main__":
    scores = []

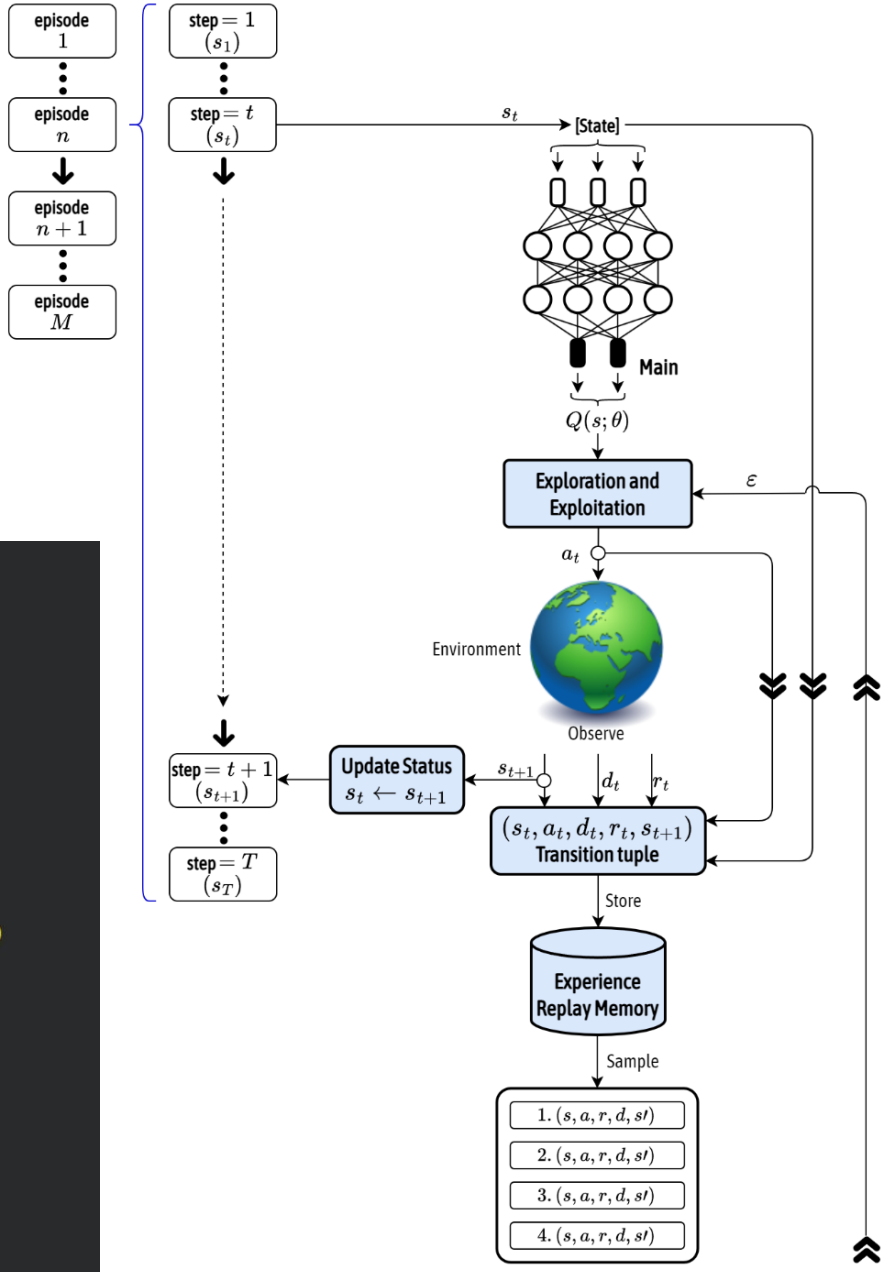
    with tqdm(total=max_episodes, desc="에피소드 진행") as pbar:
        for episode in range(max_episodes):
            state, info = agent.env.reset()
            episode_reward = 0
            terminated = False
            truncated = False

            while not (terminated or truncated):
                action = agent.get_action(state, epsilon)
                next_state, reward, terminated, truncated, _ = agent.env.step(action)
                done = terminated or truncated # 'done' 변수 정의
                agent.append_sample(state, action, reward, next_state, done)

                state = next_state
                episode_reward += reward

                if done:
                    scores.append(episode_reward)
                    pbar.set_postfix({'episode_reward': episode_reward})
                    pbar.update(1)
                    break
                if len(agent.memory) >= agent.batch_size:
                    agent.train_step()

            epsilon = min_epsilon + (max_epsilon - min_epsilon) * np.exp(-decay_rate * episode)
```



If it's time to update **then**
  For j in range(however many updates) **do** # mini-batch process
    Randomly sample a batch of transitions , $B = (s, a, r, s', d)$ from $\mathcal{D}$
    Compute targets (= expected Q)

$$y_j = Q_{expected} = \begin{cases} r_j, & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_a Q(s'_j, a_j; \theta_{targ}), & \text{otherwise} \end{cases}$$

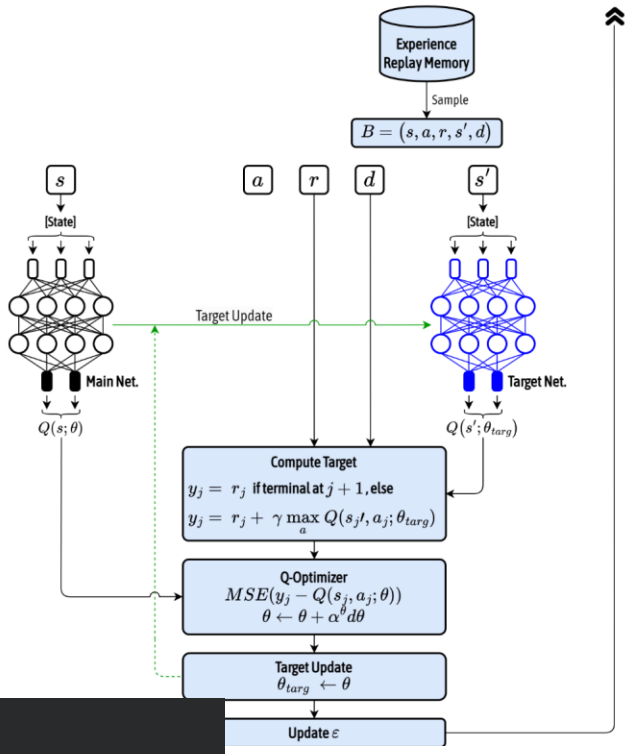
    Reset gradient $d\theta$ to 0
    Perform a gradient descent step on $(y_j - Q(s_j, a_j; \theta))^2$

$$\theta \leftarrow \theta + \alpha^{\theta} d\theta$$

    If total_step % policy_update_freq $= 0$ **then**
      Update the target networks with : # hard target update

$$\theta_{targ} \leftarrow \theta$$

    Update ε
  End For
End If
End For
End For




```python
class DQNAgent:
    def __init__(self, env: gym.Env, batch_size: int):
        self.env = env
        self.state_size = env.observation_space.shape[0]
        self.action_size = env.action_space.n

        self.batch_size = batch_size
        self.gamma = 0.99
        self.dqn = Network(self.state_size, self.action_size)
        self.optimizer = optim.Adam(self.dqn.parameters(), lr=0.001)
        self.memory = deque(maxlen=2000)

    def get_action(self, state, epsilon):
        state = torch.from_numpy(state).float().unsqueeze(0)
        q_value = self.dqn(state)[0]
        if np.random.rand() <= epsilon:
            return np.random.choice(self.action_size)
        else:
            return torch.argmax(q_value).item()

    def append_sample(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))
```

```python
def train_step(self):
    if len(self.memory) < self.batch_size:
        return

    mini_batch = random.sample(self.memory, self.batch_size)
    states, actions, rewards, next_states, dones = zip(*mini_batch)

    states = np.array(states, dtype=np.float32)
    actions = np.array(actions, dtype=np.int64)
    rewards = np.array(rewards, dtype=np.float32)
    next_states = np.array(next_states, dtype=np.float32)
    dones = np.array(dones, dtype=np.float32)

    states = torch.tensor(states, dtype=torch.float32)
    actions = torch.tensor(actions, dtype=torch.int64)
    rewards = torch.tensor(rewards, dtype=torch.float32)
    next_states = torch.tensor(next_states, dtype=torch.float32)
    dones = torch.tensor(dones, dtype=torch.float32)

    curr_Qs = self.dqn(states).gather(1, actions.unsqueeze(1)).squeeze(1)
    next_Qs = self.dqn(next_states).max(1)[0].detach()
    target_Qs = rewards + self.gamma * next_Qs * (1 - dones)

    loss = torch.nn.functional.mse_loss(curr_Qs, target_Qs)

    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
```